Laboratorium 05 – Zasoby, Czcionki i Widgety w Flutterze

Na tym laboratorium poznasz, jak dodawać własne zasoby do aplikacji (np. obrazki, czcionki), a także nauczysz się odróżniać widgety stanowe od bezstanowych. Dodatkowo omówimy rolę klasy **Scaffold**, która jest fundamentem wielu aplikacji w Flutterze.

---

1. Dodawanie zasobów (assets)

Aby użyć obrazka w aplikacji Flutter:

1. Utwórz folder `assets/images` w głównym katalogu projektu.
2. Wrzuc do niego np. plik `logo.png`.
3. Otwórz plik pubspec.yaml i dodaj wpis:

```
flutter:
  assets:
    - assets/images/logo.png
```

4. Teraz możesz wyświetlić obrazek w aplikacji:

```
Image.asset('assets/images/logo.png')
```

👉 **Wyjaśnienie:**

- `pubspec.yaml` to plik konfiguracyjny projektu Flutter.
- Dopóki nie dopiszesz tam zasobu, aplikacja nie będzie go widziała.
- `Image.asset` wczytuje grafikę lokalną, a `Image.network` może pobierać obraz z internetu.

---

2. Dodanie własnej czcionki

1. Utwórz folder `assets/fonts` i wklej czcionkę np. `CossetteTitre-Regular.ttf.` - pobraną wcześniej Google Fonts
2. W pliku pubspec.yaml dodaj:

```
flutter:
  fonts:
```

```
    - family: Cossette
      fonts:
        - asset: assets/fonts/CossetteTitre-Regular.ttf
```

3. W kodzie użyj czcionki:

```
const Text('To jest pierwszy napis', style: TextStyle(fontFamily: 'Cossette', fontSize: 18),),
```

👉 **Wyjaśnienie:**

- `family` to nazwa czcionki, którą potem podajesz w `TextStyle`.
- Pliki fontów muszą być w projekcie i zgłoszone w `pubspec.yaml`.

---

3. Stateless vs Stateful Widget

- **StatelessWidget** – nie zmienia swojego wyglądu ani danych w czasie działania (np. zwykły napis, ikona).
- **StatefulWidget** – ma stan, który można zmieniać (np. licznik, formularz, przełącznik).

Przykład Stateless:

```
class MyText extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return const Text('To jest StatelessWidget');
  }
}
```

Przykład Stateful (prosty licznik):

```
class MyCounter extends StatefulWidget {
  @override
  _MyCounterState createState() => _MyCounterState();
}

class _MyCounterState extends State<MyCounter> {
  int licznik = 0;

  @override
  Widget build(BuildContext context) {
```

```
    return Column(
      children: [
        Text('Licznik: $licznik'),
        ElevatedButton(
          onPressed: () {
            setState(() {
              licznik++;
            });
          },
          child: const Text('Dodaj +1'),
        ),
      ],
    );
  }
}
```

👉 **Różnica:** Stateless nie ma setState, a Stateful przechowuje i zmienia dane.

---

4. Klasa Scaffold

Scaffold to podstawowy układ ekranu w aplikacji Flutter. Zawiera:

- appBar – pasek u góry,
- body – główna część ekranu,
- floatingActionButton – przycisk akcji,
- drawer – menu boczne.

Przykład:

```
Scaffold(
  appBar: AppBar(title: const Text('Moja aplikacja')),
  body: const Center(child: Text('Witaj w Flutterze!')),
  floatingActionButton: FloatingActionButton(
    onPressed: () {
      print('FAB kliknięty!');
    },
    child: const Icon(Icons.add),
  ),
)
```

Zadania Praktyczne 💻

1. **Dodanie obrazka**

   Stwórz folder assets/images, dodaj tam własny obrazek i wyświetl go w aplikacji.
2. **Dodanie czcionki**

   Dodaj do projektu czcionkę z pliku .ttf i użyj jej w napisie na ekranie.
3. **Stateless vs Stateful**

   Stwórz własny StatelessWidget (np. napis) i StatefulWidget (np. licznik). Uruchom oba i porównaj zachowanie.
4. **Scaffold**

   Rozbuduj aplikację o AppBar i FloatingActionButton. Po kliknięciu w przycisk wypisz komunikat w konsoli.

Przydatne Linki 🔗

- **Dodawanie obrazków:** https://docs.flutter.dev/ui/assets/assets-and-images
- **Dodawanie czcionek:** https://docs.flutter.dev/cookbook/design/fonts
- **Stateless vs Stateful:** https://docs.flutter.dev/development/ui/interactive
- **Scaffold:** https://api.flutter.dev/flutter/material/Scaffold-class.html